BACKEND SUBMISSION · ARCHITECTURE OVERVIEW

Brand AI-Readiness Audit

An Agent Skill marketplace that audits any website, read-only, for why AI assistants can't find, read, understand, or quote the brand — and emits one evidence-backed report.

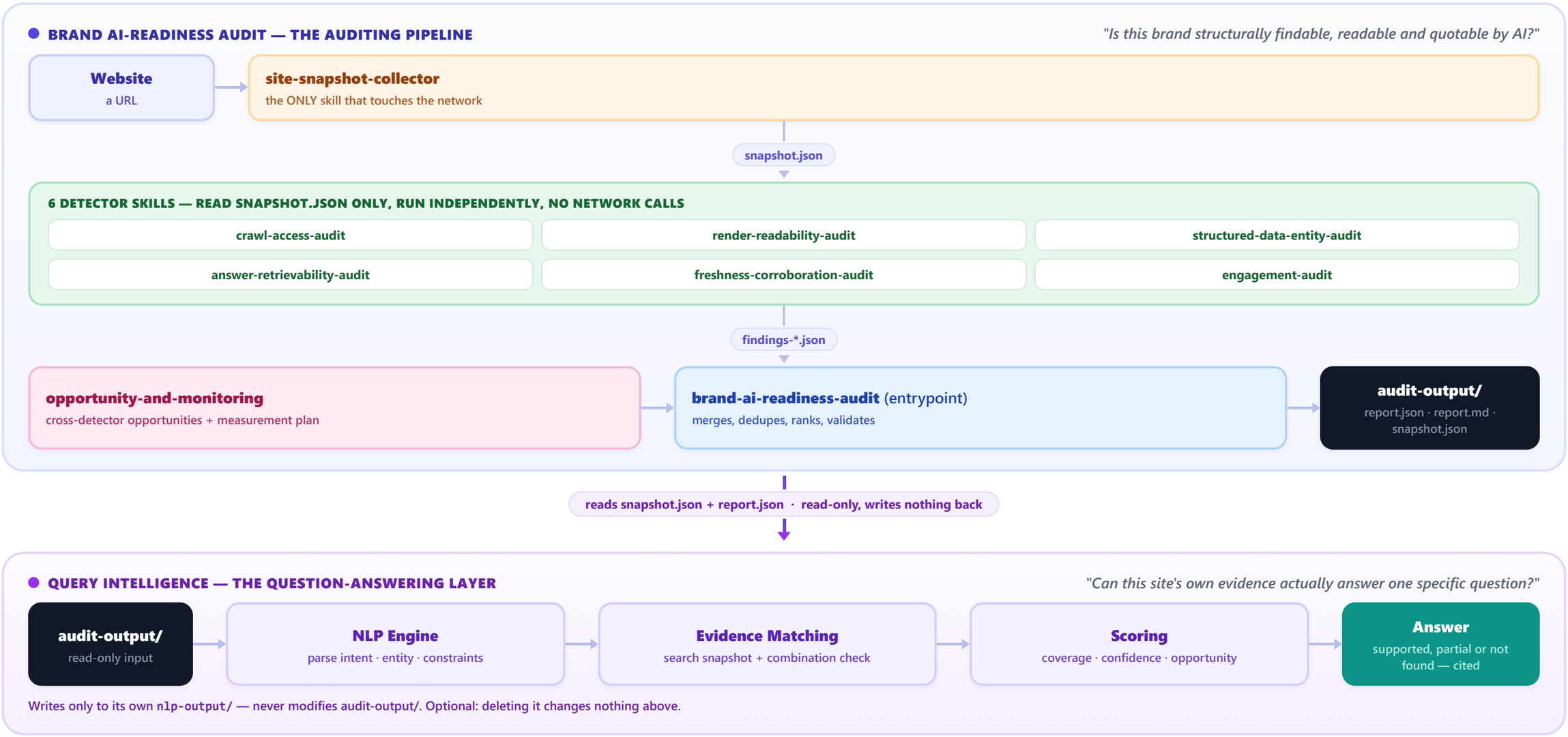
9 Agent Skills | Python 3.9+ · stdlib only | Read-only & recommend-only | Evidence-backed findings | Optional NLP Query Intelligence

This document covers the backend system only — architecture, folder layout, and how to run a complete audit from the terminal.

01 · SYSTEM ARCHITECTURE

Two pipelines, one direction of dependency

The audit finds structural gaps across a whole site. Query Intelligence then tests whether one specific question is answerable from that same evidence.



02 · WORKED EXAMPLE

What each block actually does, on one example site

A fictional store, `techmart.example`, selling laptops — the same captured evidence, read by every block below.

1 · THE AUDIT, BLOCK BY BLOCK

SNAPSHOT `site-snapshot-collector` visits `/laptops/dell-xps-13` and saves the raw HTML a crawler sees, the JavaScript-rendered DOM, the page's JSON-LD data, robots.txt rules, headers and links. Every block below reads only this saved copy — nothing is fetched twice.

crawl-access-audit — NO DEFECT
`/cart` is correctly noindexed; product pages stay open to GPTBot.

render-readability-audit — MEDIUM
Battery life only appears after JavaScript runs — missing from the raw HTML.

structured-data-entity-audit — HIGH
JSON-LD price ₹58,999 contradicts the visible page text, ₹54,999.

answer-retrievability-audit — NO DEFECT
"16GB RAM" sits in a clean spec table near the top of the page.

freshness-corroboration-audit — MEDIUM
The category listing still shows last month's ₹54,999 price.

engagement-audit — LOW
The comparison page has no link onward — a dead end.

How the report gets built: each block above writes its own `findings-*.json`, independently and without seeing the others' output. `opportunity-and-monitoring` then looks across all of them for patterns — "3 product pages share this exact defect" becomes one opportunity, not three separate findings. The entrypoint merges everything, drops exact duplicates, scores each finding's severity from *how much it blocks the funnel* × *how many pages* × *how sure we are*, ranks them, and writes `report.json` (data) + `report.md` (read this). A check that genuinely couldn't run is marked **not assessed** — never guessed, never silently dropped.

2 · QUERY INTELLIGENCE — SAME SITE, ONE SPECIFIC QUESTION

INPUT — A PLAIN-ENGLISH QUESTION, NOTHING ELSE

"Best laptop for CSE students under ₹60,000 with 16GB RAM"

1 · Parse
Breaks the question into requirements: entity = laptop, price ≤ ₹60,000, RAM = 16GB, audience = CSE students.

2 · Match
Searches the same snapshot.json for each requirement — only within pages that are actually about "laptop".

3 · Combine-check
Confirms one single page satisfies every requirement together — not different products stitched into one answer.

entity: laptop SUPPORTED
dell-xps-13 page

RAM: 16GB SUPPORTED
found in the spec table

price ≤ ₹60,000 CONTRADICTED
₹58,999 in data vs. ₹64,999 on page

audience: CSE students NOT FOUND
no audience signal on this page

Output — Coverage 46% · Answerability 100% · Confidence high · Opportunity: fix the price mismatch and this exact query would score far higher.

03 · REAL OUTPUT

Not a mockup — an actual audit run

This exact `report.json` is checked into the repo at `brand-ai-readiness-audit/audit-output/`, produced by running `python audit.py https://adobe.com`.

752	25	11	0 / 0	5 / 6	159.8s
PAGES DISCOVERED	PAGES SAMPLED	FINDINGS	CRITICAL / HIGH	MEDIUM / LOW	RUNTIME

3 OF THE 11 REAL FINDINGS

`F-001 · SDE.NO_SAMEAS · structured-data-entity-audit` — MEDIUM

The organization entity has no identity links

"The Organization node on the homepage declares no `sameAs` property — no links to independent profiles a system could use to confirm this is the same real-world brand." → **fix: add sameAs links to profiles the brand controls.**

`F-002 · FRC.DATE_SIGNAL_CONFLICT · freshness-corroboration-audit` — MEDIUM

A page's own date signals disagree with each other

"24 pages carry date signals that disagree by more than 2 days. Example: sitemap says 2026-08-19, the server's Last-Modified header says 2026-09-09." → **fix: drive every date signal from the same publish event.**

`F-006 · ACC.SITEMAP_LASTMOD_UNIFORM · crawl-access-audit` — LOW

Every sitemap entry carries the same modification date

"home-sitemap.xml lists 66 URLs and exactly one distinct lastmod value across all of them — the signature of a build timestamp, not real content changes." → **fix: emit lastmod from real publish time.**

Every finding in the real file also carries `affected_urls` (every failing page), `evidence_detail` (exact method + observed value), and a full `suggested_action` — this is a two-line summary of each.

THE REAL REPORT.JSON SHAPE

```
{
  "site": "adobe.com",
  "audited_at": "2026-09-12T09:22:58Z",
  "summary": {
    "total_findings": 11, "critical": 0,
    "high": 0, "medium": 5, "low": 6
  },
  "coverage": {
    "pages_discovered": 752,
    "pages_sampled": 25,
    "not_assessed": [ 4 items, each with a reason ]
  },
  "findings": [
    {
      "id": "F-001",
      "check_id": "SDE.NO_SAMEAS",
      "severity": "medium",
      "evidence": "... the exact quote ...",
      "affected_urls": [ every failing page ],
      "suggested_action": { summary, how, owner, effort }
      // 10 more findings
    }
  ],
  "opportunities": [ 7 cross-detector patterns ],
  "action_sequence": [ 3 priority bands ]
}
```

04 · REPOSITORY LAYOUT

Folder structure & where the report lands

Backend only — everything below lives under `brand-ai-readiness-audit/`

```
brand-ai-readiness-audit/
├── marketplace.json # manifest: 9 skills, 1 entrypoint
├── audit.py # CLI — runs the full pipeline
├── skills/ # the 9 Agent Skills
│   ├── brand-ai-readiness-audit/ (entrypoint)
│   ├── site-snapshot-collector/
│   ├── crawl-access-audit/
│   ├── render-readability-audit/
│   ├── structured-data-entity-audit/
│   ├── answer-retrievability-audit/
│   ├── freshness-corroboration-audit/
│   ├── engagement-audit/
│   └── opportunity-and-monitoring/
├── tools/ # validator + run helpers
├── tests/ # 35 fixtures + test harness
├── nlp-query-intelligence/ # optional Query Intelligence subsystem
└── audit-output/ ← every audit run writes here
```

→

audit-output/

Every command below writes into this one folder. Re-running overwrites it, unless `--out DIR` is passed.

- snapshot.json
- report.json
- report.md
- SUMMARY.md
- findings-*.json ×6
- opportunities.json

report.md — read this first, human-readable.
report.json — same data, schema-validated, machine-readable.
snapshot.json — the raw evidence every finding was derived from.
SUMMARY.md — 5–6 line plain-English summary of the run.

05 · RUNNING IT — BACKEND TERMINAL ONLY

From a URL to a report, in one command

No frontend, no browser UI required. Everything below is a plain terminal command run from inside `brand-ai-readiness-audit/`.

1 Run a full audit

```
cd brand-ai-readiness-audit
python audit.py https://example.com
```

Writes **`snapshot.json`**, **`report.json`** and **`report.md`** into **`audit-output/`** — overwrites the previous run.

2 Keep every run — name the folder yourself (optional)

```
python audit.py https://flipkart.com --out audit-output/flipkart
python audit.py https://amazon.in    --out audit-output/amazon
```

Same command, one added flag. Each site gets its own named subfolder under **`audit-output/`** — nothing gets overwritten.

3 Same, plus a short plain-English summary (optional)

```
python tools/run_audit_with_summary.py https://example.com
```

Runs audit.py, then writes **`audit-output/SUMMARY.md`** — a 5–6 line result anyone can read at a glance. Accepts `--out` too.

4 Ask the site a specific question (optional — Query Intelligence)

```
cd nlp-query-intelligence
python pipeline.py test --query "Best laptop under $1000" --snapshot ../audit-output/snapshot.json \
  --report ../audit-output/report.json --out nlp-output/result.json
```

Checks whether the site's own already-captured evidence could support that answer.

5 Verify everything still works

```
python tools/validate_marketplace.py --root . --strict   # Agent Skills spec conformance
python tests/run_tests.py                          # fixtures, scoring, determinism
```


Read-only · No network access beyond the collector · No external LLM